

```
1 def knapsack(weights, values, capacity):
7     for i in range(1, n + 1):
8         for w in range(capacity + 1):
10             if weights[i - 1] <= w:
14                 )
15             else:
16                 dp[i][w] = dp[i - 1][w]
17
18     return dp[n][capacity]
19
20
21 weights = [1, 3, 4, 5]
22 values = [1, 4, 5, 7]
23 capacity = 7
24
25 print("Maximum Profit:", knapsack(weights, values, capacity))
```

Maximum Profit: 9

=== Code Execution Successful ===

```
3 def tsp(graph):
7     def visit(mask, pos):
21         return ans
22
23     return visit(1, 0)
24
25
26
27 graph = [
28     [0, 10, 15, 20],
29     [10, 0, 35, 25],
30     [15, 35, 0, 30],
31     [20, 25, 30, 0]
32 ]
33
34 print("Minimum Cost:", tsp(graph))
```

Minimum Cost: 80

=== Code Execution Successful ===

```
1 def optimal_bst(keys, freq):
10     for length in range(2, n + 1):
12         for i in range(n - length + 1):
19             for r in range(i, j + 1):
24                 cost[i][j] = min(
25                     cost[i][j],
26                     left + right + total_freq
27                 )
28
29     return cost[0][n - 1]
30
31
32 keys = [10, 20, 30]
33 freq = [34, 8, 50]
34
35 print("Optimal BST Cost:", optimal_bst(keys, freq))
```

Optimal BST Cost: 142

=== Code Execution Successful ===

```
1 def knapsack(wt, val, W):
7     for i in range(1, n + 1):
9         for w in range(W + 1):
11             if wt[i - 1] <= w:
17                 )
18             else:
19                 dp[i][w] = dp[i - 1][w]
20
21     return dp[n][W]
22
23
24 wt = [1, 3, 4, 5]
25 val = [1, 4, 5, 7]
26 W = 7
27
28 print("Maximum Profit =", knapsack(wt, val, W))
```

Maximum Profit = 9

=== Code Execution Successful ===

```
1 def knapsack(W, wt, val, n, memo):
36
37     return memo[n][W]
38
39
40 wt = [1, 3, 4, 5]
41 val = [1, 4, 5, 7]
42 W = 7
43
44 memo = [[-1] * (W + 1)
45         for _ in range(len(wt) + 1)]
46
47 print(
48     "Maximum Profit =",
49     knapsack(W, wt, val, len(wt), memo)
50 )
```

Maximum Profit = 9

=== Code Execution Successful ===

```
1 def subset_sum(arr, target):
11     for i in range(1, n + 1):
13         for j in range(1, target + 1):
14
15             if arr[i - 1] <= j:
16                 dp[i][j] = (
17                     dp[i - 1][j] or
18                     dp[i - 1][j - arr[i - 1]]
19                 )
20             else:
21                 dp[i][j] = dp[i - 1][j]
22
23     return dp[n][target]
24
25
26 print(subset_sum([3, 34, 4, 12, 5, 2], 9))
```

True

=== Code Execution Successful ===

```

14 def tsp(mask, pos):
20
21     for city in range(n):
22
23         if not mask & (1 << city):
24
25             ans = min(
26                 ans,
27                 graph[pos][city] +
28                 tsp(mask | (1 << city), city)
29             )
30
31     return ans
32
33
34 print("Minimum Cost =", tsp(1, 0))

```

Minimum Cost = 80

=== Code Execution Successful ===

```
1 def tsp_dp(graph):
29     for i in range(1, n):
31         ans = min(
34             )
35
36     return ans
37
38
39 graph = [
40     [0, 10, 15, 20],
41     [10, 0, 35, 25],
42     [15, 35, 0, 30],
43     [20, 25, 30, 0]
44 ]
45
46 print("Minimum Cost =", tsp_dp(graph))
```

Minimum Cost = 80

=== Code Execution Successful ===



```

1  def optimal_bst(freq):
10     for L in range(2, n + 1):
12         for i in range(n - L + 1):
20             for r in range(i, j + 1):
23
24                 right = cost[r + 1][j] if r < j else 0
25
26                 cost[i][j] = min(
27                     cost[i][j],
28                     left + right + total
29                 )
30
31     return cost[0][n - 1]
32
33
34 print("Optimal Cost =", optimal_bst([34, 8, 50]))

```

Optimal Cost = 142

=== Code Execution Successful ===

```
1 def matrix_chain(arr):
2     for L in range(2, n):
3         for i in range(1, n - L + 1):
4             for k in range(i, j):
5                 dp[i][j] = min(
6                     dp[i][k],
7                     dp[k+1][j],
8                     q
9                 )
10
11     return dp[1][n - 1]
12
13 print(
14     "Minimum Multiplications =",
15     matrix_chain([10, 20, 30, 40, 30])
16 )
```

Minimum Multiplications = 30000

=== Code Execution Successful ===

```
1 def min_coins(coins, amount):
7     for i in range(1, amount + 1):
9         for c in coins:
11            if c <= i:
13                dp[i] = min(
14                    dp[i],
15                    dp[i - c] + 1
16                )
17
18     return dp[amount]
19
20
21 print(
22     "Minimum Coins =",
23     min_coins([1, 2, 5], 11)
24 )
```

Minimum Coins = 3

=== Code Execution Successful ===

```
1 def rod_cut(price, n):
2     for i in range(1, n + 1):
3         for j in range(i):
4             dp[i] = max(
5
6             )
7
8     return dp[n]
9
10
11
12
13
14
15
16
17 print(
18     "Maximum Revenue =",
19     rod_cut(
20         [1, 5, 8, 9, 10, 17, 17, 20],
21         8
22     )
23 )
```

Maximum Revenue = 22

=== Code Execution Successful ===

```
1  def min_cost(cost):
22      for i in range(1, m):
24          for j in range(1, n):
26              dp[i][j] = (
32          )
33
34      return dp[m - 1][n - 1]
35
36
37  cost = [
38      [1, 3, 1],
39      [1, 5, 1],
40      [4, 2, 1]
41  ]
42
43  print("Minimum Cost =", min_cost(cost))
```

Minimum Cost = 7

=== Code Execution Successful ===